

# Informe Técnico: Algoritmos y Estructuras de Datos

Análisis de Implementación, Operaciones Fundamentales y Manipulación de Arreglos Unidimensionales en Java

PROYECTO  
ArregloEstadistico.java

ESTRUCTURA  
Arreglo Unidimensional

ESTADO  
Ejecución Verificada (8/8)

**Resumen Ejecutivo:** El presente documento evalúa las 8 operaciones fundamentales sobre arreglos estructurados utilizando el atributo control de capacidad tope. Se detalla el algoritmo paso a paso, la explicación técnica del código orientado a objetos y la evidencia empírica con datos de prueba reales.

## 1. ELIMINAR UN ELEMENTO "X"

### Opción 4 Método para eliminar la primera aparición de un valor x

#### Algoritmo Paso a Paso

- Comprobar que el arreglo exista y contenga datos registrados (tope > 0).
- Buscar la posición del valor x utilizando búsqueda secuencial. Si no existe (-1), retornar.
- Mover los elementos desde pos + 1 hasta tope - 1 una casilla a la izquierda.
- Reducir el atributo tope en 1 unidad.

#### Explicación del Código

Se localiza el índice de x. El bucle for (int i = pos; i < tope - 1; i++) desplaza los elementos adyacentes hacia la izquierda para eliminar el espacio vacío. Finalmente, setTope(getTope() - 1) actualiza el límite operativo sin necesidad de reasignar el tamaño físico del arreglo.

| Datos Iniciales          | Entrada Usuario | Resultado Esperado    | Resultado Obtenido    | ¿Coincide? |
|--------------------------|-----------------|-----------------------|-----------------------|------------|
| [5, 12, 7, 20, 3, 8, 15] | x = 7           | [5, 12, 20, 3, 8, 15] | [5, 12, 20, 3, 8, 15] | Sí         |

## 2. MODIFICAR UN ELEMENTO "X"

### Opción 5 Método para reemplazar el valor x por un nuevo valor

#### Algoritmo Paso a Paso

- Verificar que el arreglo no esté vacío.
- Obtener la posición de la primera ocurrencia de x mediante buscar(x).
- Si el elemento existe (pos != -1), sobrescribir esa posición con nuevo.
- Si no se encuentra, emitir mensaje de advertencia y finalizar.

#### Explicación del Código

El método solicita el valor buscado x y el sustituto nuevo. La modificación se realiza de manera directa mediante getNumerito()[pos] = nuevo en \$O(1)\$ tras la búsqueda. El tamaño del arreglo y el valor de tope permanecen inalterados.

### 3. INSERTAR UN ELEMENTO "X" EN UNA POSICIÓN "Y"

#### Opción 6 Inserción con desplazamiento a la derecha

##### Algoritmo Paso a Paso

1. Comprobar la existencia de espacio disponible ( $\text{tope} < \text{capacidad}$ ).
2. Validar que la posición solicitada y cumpla  $\$1 \leq y \leq \text{tope} + 1$ .
3. Desplazar elementos de derecha a izquierda desde  $\text{tope}$  hasta  $y$ .
4. Asignar el valor en  $y - 1$  e incrementar  $\text{tope}$  en 1.

##### Explicación del Código

La posición  $y$  ingresada por el usuario (base 1) se convierte a base 0 ( $y - 1$ ). El ciclo recorre en sentido decreciente ( $i--$ ) copiando  $a[i] = a[i-1]$  para liberar la casilla de inserción sin perder datos. Soporta sobrecarga para inserción al final.

| Datos Iniciales          | Entrada Usuario | Resultado Esperado           | Resultado Obtenido           | ¿Coincide? |
|--------------------------|-----------------|------------------------------|------------------------------|------------|
| [5, 12, 7, 20, 3, 8, 15] | x = 99, y = 3   | [5, 12, 99, 7, 20, 3, 8, 15] | [5, 12, 99, 7, 20, 3, 8, 15] | Sí         |

### 4. SUMAR LOS ELEMENTOS DEL ARREGLO

#### Opción 7 Acumulación aritmética total del arreglo

##### Algoritmo Paso a Paso

1. Comprobar que el arreglo contenga datos válidos ( $\text{exigeDatos}()$ ).
2. Inicializar una variable acumuladora  $\text{suma} = 0$ .
3. Iterar mediante un bucle for desde la casilla 0 hasta  $\text{tope} - 1$ .
4. Retornar el acumulado global para su posterior presentación.

##### Explicación del Código

El método está especificado en la interfaz `Calculable` e implementado en `ArregloEstadistico`. Garantiza la integridad del cálculo iterando únicamente hasta  $\text{tope}$  e ignorando posiciones vacías reservadas en memoria.

| Datos Iniciales          | Entrada Usuario | Resultado Esperado | Resultado Obtenido | ¿Coincide? |
|--------------------------|-----------------|--------------------|--------------------|------------|
| [5, 12, 7, 20, 3, 8, 15] | Opción 7        | 70                 | 70                 | Sí         |

## 5. NÚMERO MENOR, NÚMERO MAYOR Y PROMEDIO

### Opción 8 Estadísticos descriptivos del arreglo

#### Algoritmo Paso a Paso

1. Validar presencia de datos. Inicializar menor y mayor con `a[0]`.
2. Iterar desde `i = 1` hasta `tope - 1` comparando cada valor para actualizar los candidatos.
3. Calcular el promedio dividiendo el acumulado total entre `tope`.
4. Formatear la salida a dos decimales y mostrar resultados.

#### Explicación del Código

Se inicializan los extremos con el primer valor para soportar números negativos correctamente. El cálculo del promedio aplica un casteo explícito (`double`) `sumar() / getTope()` para prevenir la división entera de Java.

| Datos Iniciales          | Entrada Usuario | Resultado Esperado               | Resultado Obtenido               | ¿Coincide? |
|--------------------------|-----------------|----------------------------------|----------------------------------|------------|
| [5, 12, 7, 20, 3, 8, 15] | Opción 8        | Menor: 3, Mayor: 20, Prom: 10.00 | Menor: 3, Mayor: 20, Prom: 10.00 | Sí         |

## 6. SUMA DE LOS ELEMENTOS IMPARES

### Opción 9 Acumulación selectiva bajo condición de imparidad

#### Algoritmo Paso a Paso

1. Validar la existencia de elementos en la estructura.
2. Inicializar el acumulador `suma = 0`.
3. Recorrer el arreglo evaluando la condición de número impar: `a[i] % 2 != 0`.
4. Si cumple la condición, acumular el valor y retornar el total.

#### Explicación del Código

La evaluación utiliza de forma estricta `!= 0` en lugar de `== 1`. En Java, la operación residuo sobre números impares negativos produce `-1`, por lo que esta validación contempla el universo completo de los enteros.

| Datos Iniciales          | Entrada Usuario | Resultado Esperado  | Resultado Obtenido | ¿Coincide? |
|--------------------------|-----------------|---------------------|--------------------|------------|
| [5, 12, 7, 20, 3, 8, 15] | Opción 9        | 30 (5 + 7 + 3 + 15) | 30                 | Sí         |

## 7. MOVER EL PRIMER ELEMENTO A LA ÚLTIMA POSICIÓN

### Opción 10 Rotación circular hacia la izquierda

#### Algoritmo Paso a Paso

1. Respalidar el primer elemento  $a[0]$  en una variable auxiliar.
2. Desplazar los elementos hacia la izquierda:  
 $a[i] = a[i + 1]$ .
3. Escribir el valor respaldado en la última casilla activa ( $\text{tope} - 1$ ).

#### Explicación Técnica

Preserva el primer valor antes del desplazamiento. El ciclo recorre hasta  $\text{tope} - 2$  evitando desbordamiento de índices.

| Datos Iniciales          | Entrada Usuario | Resultado Esperado / Obtenido | ¿Coincide? |
|--------------------------|-----------------|-------------------------------|------------|
| [5, 12, 7, 20, 3, 8, 15] | Opción 10       | [12, 7, 20, 3, 8, 15, 5]      | Sí         |

## 8. MOVER EL PRIMER ELEMENTO PAR AL FINAL

### Opción 11 Búsqueda condicional y desplazamiento al final

#### Algoritmo Paso a Paso

1. Buscar el primer elemento par ( $a[i] \% 2 == 0$ ) y guardar su posición.
2. Si se encuentra, respaldar el número par en una variable temporal.
3. Desplazar a la izquierda desde la posición hallada hasta  $\text{tope} - 1$ .
4. Escribir el valor en la posición final  $a[\text{tope} - 1]$ .

#### Explicación Técnica

Combina la búsqueda lineal corta (detenida con break al primer acierto) con una reestructuración de arreglo equivalente a la operación 7.

| Datos Iniciales          | Entrada Usuario | Resultado Esperado / Obtenido | ¿Coincide? |
|--------------------------|-----------------|-------------------------------|------------|
| [5, 12, 7, 20, 3, 8, 15] | Opción 11       | [5, 7, 20, 3, 8, 15, 12]      | Sí         |

## CONCLUSIONES Y DISEÑO ORIENTADO A OBJETOS

- **Manejo del Atributo Tope:** La administración explícita de un puntero de tamaño activo ( $\text{tope}$ ) permite operar sobre arreglos estáticos de Java de forma dinámica sin incurrir en realocación constante de memoria.
- **Sentido del Desplazamiento:** En operaciones de inserción, el recorrido debe realizarse de derecha a izquierda ( $i--$ ) para evitar sobrescribir datos, mientras que en eliminación/rotación debe ejecutarse de izquierda a derecha ( $i++$ ).
- **Arquitectura Modular:** La separación de responsabilidades entre la clase abstracta `ArregloBase`, la lógica de edición en `Arreglos` y los cálculos en `ArregloEstadistico` asegura un código mantenible, extensible y testeable.